Market Intelligence Platform — Architecture Reference

> ***Last updated:*** *March 2026*
> ***Branch baseline:*** `main` *(domain modeling fully merged)*

Table of Contents

1. System Overview
2. Repository Layout
3. Domain Layer
4. Application Layer (Adapters)
5. API Layer
6. Frontend
7. Data Sources
8. Testing Strategy
9. CI/CD Pipeline
10. AWS Infrastructure
11. Security & Auth
12. Branching Strategy
13. Key Design Decisions

---

1. System Overview

The Market Intelligence Platform is a professional-grade multi-timeframe trading analysis system. It ingests OHLCV market data, runs a full analytical pipeline across four timeframes, and delivers structured trade signals, expert battle plans, and risk intelligence to traders through a real-time Angular dashboard.

```
┌──────────────────────────────────────────────────────────────────┐
│                        Angular SPA (S3 + CloudFront)             │
│  Market Heatmap · Expert Battle Plan · Signal Intelligence       │
└──────────────────────────┬───────────────────────────────────────┘
                           │ HTTPS / REST
┌──────────────────────────▼───────────────────────────────────────┐
│            API Gateway  →  AWS Lambda (FastAPI + Mangum)         │
│                      backend/app/main.py                         │
└────┬──────────────────────┬──────────────────────────────────┬───┘
     │ Analyzers            │ Signal Generator                 │ Auth
     ▼                      ▼                                  ▼

┌────────────┐     ┌──────────────────┐              ┌──────────────┐
│  Analyzer  │     │  Domain Layer    │              │ Google OAuth │
│  Adapters  │──▶│  (Pure Python)   │              │  DynamoDB    │
│ (app/      │     │  domain/         │              │  Sessions    │
│ analyzers/)│     │  indicators/     │              └──────────────┘
└────────────┘     │  levels/         │
                   │  signals/        │
                   │  trading/        │
                   │  constants.py    │
                   └──────────────────┘
                           │
                   ┌───────▼──────────┐
```

```
                  │  TwelveData    │
                  │  Market API    │
                  └────────────────┘
```

---

2. Repository Layout

```
market_analyser_ai/
│
├── backend/                        # Python FastAPI backend
│   ├── app/
│   │   ├── main.py                 # FastAPI app + analyze_instrument_lazy()
│   │   ├── models.py               # Pydantic response models (data contract)
│   │   ├── signal_generator.py     # Composite signal assembly (thin adapter)
│   │   ├── auth.py                 # JWT session management
│   │   ├── oauth.py                # Google OAuth flow
│   │   ├── db.py                   # DynamoDB user store
│   │   ├── config_loader.py        # instruments.yaml + StrategySettings
│   │   ├── data_fetcher.py         # TwelveData API wrapper
│   │   └── analyzers/
│   │       ├── trend_analyzer.py
│   │       ├── pullback_analyzer.py
│   │       ├── strength_analyzer.py
│   │       ├── volatility_analyzer.py
│   │       ├── technical_analyzer.py
│   │       ├── signal_generator.py
│   │       ├── day_trading_expert.py
│   │       ├── position_sizer.py
│   │       ├── pullback_warning_analyzer.py
│   │       ├── fundamentals_analyzer.py
│   │       ├── intermarket_analyzer.py
│   │       ├── relative_strength_analyzer.py
│   │       ├── session_vwap_analyzer.py
│   │       ├── volume_profile_analyzer.py
│   │       ├── liquidity_map_analyzer.py
│   │       ├── block_flow_analyzer.py
│   │       ├── backtest_engine.py
│   │       └── geo_risk_analyzer.py
│   │
│   ├── domain/                     # ★ Domain layer — single source of truth
│   │   ├── constants.py            # All thresholds, weights, magic numbers
│   │   ├── indicators/             # Pure indicator math
│   │   │   ├── rsi.py
│   │   │   ├── atr.py
│   │   │   ├── adx.py
│   │   │   ├── vwap.py
│   │   │   ├── macd.py
│   │   │   └── bollinger.py
│   │   ├── levels/                 # Price level calculations
│   │   │   ├── pivot_points.py
│   │   │   ├── fibonacci.py
```

```
│   │   │   ├── std_bands.py
│   │   │   ├── breakout.py
│   │   │   ├── support_resistance.py
│   │   │   └── linear_regression.py
│   │   ├── signals/                # Trade signal logic
│   │   │   ├── scoring_engine.py
│   │   │   ├── filter_engine.py
│   │   │   └── conflict_detector.py
│   │   └── trading/                # Intraday constructs
│   │       ├── rvol.py
│   │       ├── opening_range.py
│   │       └── position_sizer.py
│   │
│   ├── tests/
│   │   ├── conftest.py             # Shared fixtures (mock data, Pydantic models)
│   │   ├── test_analyzers.py       # JSON serialisation compliance tests
│   │   ├── test_calculations.py    # Expert Battle Plan + calculation logic
│   │   ├── test_signal_generator.py
│   │   ├── test_pipeline_integration.py
│   │   ├── test_expert_plan.py
│   │   ├── test_liquidity_map.py
│   │   └── domain/                 # ★ Domain unit tests (249 tests)
│   │       ├── conftest.py
│   │       ├── test_indicators.py
│   │       ├── test_levels.py
│   │       ├── test_signals.py
│   │       └── test_trading.py
│   │
│   ├── config/
│   │   └── instruments.yaml        # Watchlist + analysis parameters
│   ├── Dockerfile                  # Multi-stage Docker image (Lambda-compatible)
│   ├── requirements.txt
│   └── pytest.ini
│
├── frontend/                       # Angular 17 SPA
│   └── src/app/
│       ├── app.ts / app.html       # Root component + heatmap
│       └── instrument-card/        # Per-instrument analysis card
│
├── infrastructure/
│   └── terraform/                  # Full IaC for AWS resources
│       ├── main.tf                 # S3 backend, provider config
│       ├── lambda.tf               # Lambda function + API Gateway
│       ├── s3_cloudfront.tf        # Frontend hosting + CDN
│       ├── ecr.tf                  # Container registry
│       ├── dynamodb.tf             # User sessions table
│       ├── iam.tf                  # Roles + GitHub OIDC trust
│       ├── github_oidc.tf          # Keyless CI/CD auth
│       ├── variables.tf
│       └── outputs.tf
│
├── tests-e2e/
```

```
│   └── critical-flows.spec.ts      # Playwright end-to-end tests
│
└── .github/workflows/
    ├── ci.yml                      # PR validation (lint, tests, Docker build)
    └── deploy.yml                  # Deploy to DEV (develop) / PROD (main)
```

---

3. Domain Layer

The domain layer is the **architectural centrepiece** introduced in the `domainmodeling` refactor. It centralises all computational logic as pure Python functions with no framework dependencies.

Design Rules

Rule	Rationale
No pandas inside domain	Enables unit testing without DataFrames; functions are composable
No Pydantic models inside domain	Decouples business logic from API schema; domain is independently reusable
Primitive inputs only	`list[float]`, `float`, `int`, `bool` — easy to test, easy to reason about
Dataclasses for structured output	`BollingerResult`, `MACDResult`, `PivotPoints`, etc. — typed but lightweight
All magic numbers in `constants.py`	Single place to adjust thresholds for all strategies

Sub-packages

`domain/indicators/`

Pure technical indicator implementations.

Module	Functions	Notes
`rsi.py`	`calculate_rsi()`, `classify_rsi()`, `detect_rsi_divergence()`	Wilder smoothing
`atr.py`	`calculate_atr()`, `calculate_atr_series()`	True range via EMA
`adx.py`	`calculate_adx()`, `classify_adx()`	Directional movement index
`vwap.py`	`calculate_vwap()`, `classify_vwap_position()`	VWAP + distance %
`macd.py`	`calculate_macd()`, `detect_histogram_weakening()`	Returns `MACDResult` dataclass
`bollinger.py`	`calculate_bollinger_bands()`, `is_band_reentry()`	Returns `BollingerResult`

`domain/levels/`

Price-level and structure calculations.

Module	Functions
`pivot_points.py`	`calculate_pivot_points()` → `PivotPoints` (P, S1, S2, S3, R1, R2, R3)

`fibonacci.py`	`calculate_fibonacci_levels()` → `FibonacciLevels` (23.6%, 38.2%, 50%, 61.8%, extensions)
`std_bands.py`	`calculate_std_dev_bands()`, `calculate_rolling_std()`
`breakout.py`	`detect_donchian_breakout()` → `BreakoutResult`
`support_resistance.py`	`find_swing_lows()`, `find_swing_highs()`, `nearest_support_below()`
`linear_regression.py`	`calculate_linear_regression_slope()`, `classify_slope()`

`domain/signals/`

Trade signal assembly logic.

Module	Functions	Purpose
`scoring_engine.py`	`compute_trend_score()`, `compute_pullback_score()`, `compute_strength_score()`, `compute_composite_score()`, `classify_recommendation()`	Weighted scoring → -100..+100
`filter_engine.py`	`apply_adx_filter()`, `apply_benchmark_filter()`, `apply_candle_filter()`, `apply_macro_shield()`, `apply_relative_strength_filter()`, `apply_all_hard_filters()`	Hard filters that can block a trade signal
`conflict_detector.py`	`detect_adx_direction_mismatch()`, `detect_mtf_disagreement()`, `detect_signal_conflicts()`	Identifies contradictions between timeframes

`domain/trading/`

Intraday-specific constructs.

Module	Functions
`rvol.py`	`calculate_rvol()`, `classify_rvol()`, `is_high_intent()`
`opening_range.py`	`detect_opening_range()` → `ORBData`, `classify_orb_context()`
`position_sizer.py`	`calculate_correlation_penalty()`, `calculate_risk_amount()`, `calculate_risk_per_unit()`, `calculate_position_units()`

`domain/constants.py`

Single location for all thresholds, weights, and parameters:

```
SIGNAL_CONVICTION_THRESHOLD = 50       # Min composite score for trade-worthy
FILTER_ADX_THRESHOLD        = 15       # Min ADX before hard-filter blocks trade
RVOL_LOOKBACK_DAYS          = 20       # Days used for average volume baseline
INDICATOR_RSI_OVERBOUGHT    = 70
INDICATOR_RSI_OVERSOLD      = 30
INDICATOR_LRL_SLOPE_THRESHOLD = 0.001  # Flat vs trending slope boundary
PULLBACK_WARNING_THRESHOLD  = 3        # Min score to trigger pullback warning
# ... 30+ additional constants
```

---

4. Application Layer (Adapters)

The `app/analyzers/` modules are **thin adapters**. They:

1. Receive pandas DataFrames from the data fetcher
2. Extract primitive arrays (`.tolist()` , `.iloc[-1]` , etc.)
3. Call domain functions
4. Map results back into Pydantic response models

```
# Example: strength_analyzer.py (adapter pattern)
from domain.indicators.rsi import calculate_rsi as _domain_rsi

def analyze_daily_strength(df: pd.DataFrame, ...) -> StrengthAnalysis:
    closes = df['Close'].tolist()          # DataFrame → list[float]
    rsi = _domain_rsi(closes, period=14)   # Pure domain call
    return StrengthAnalysis(rsi=rsi, ...)  # Pydantic model output
```

Analysis Pipeline (per instrument)

`analyze_instrument_lazy()` in `main.py` orchestrates the full pipeline in this order:

```
1. Fetch OHLCV data      TwelveData API (macro/pullback/execution/expert timeframes)
2. Monthly Trend         20/50 MA crossover → TrendAnalysis
3. Weekly Pullback       Swing lows + support proximity → PullbackAnalysis
4. Daily Strength        RSI + ADX + VWAP + volume → StrengthAnalysis
5. Market Phase          Phase classification (markup/distribution/etc.) → PhaseAnalysis
6. Volatility & Risk     ATR + ATR percentile rank + volatility regime → VolatilityAnalysis
7. Technical Indicators  Pivot Points + Fibonacci + Std Bands + Breakout + LR Slope
8. Candle Patterns       Bullish/bearish engulfing, doji, shooting star, hammer
9. Fundamentals          Economic calendar events (FMP API)
10. Backtest Results     Win rate, Sharpe, max drawdown, MAE, expectancy
11. Relative Strength    Correlation vs benchmark (SPX/DXY/TNX)
12. Intermarket Context  DXY + US10Y direction → Gold implication
13. Session Context      PDH, PDL, London Open
14. RVOL + ORB           Opening range breakout + relative volume
15. Expert Battle Plan   6-section battle plan
(Situation/Entry/Targets/Stop/Conviction/Context)
16. Signal Generator     Composite score + hard filters + conflict detection → TradeSignal
17. Position Sizing      Correlation-adjusted units + risk-reward
18. Session VWAP         Asia/London/NY session anchored VWAP ± 1σ
19. Volume Profile       POC + VAH + VAL (20 buckets intraday, 50 daily)
20. Liquidity Map        Swing highs/lows + round-number liquidity clusters
21. Block Flow           Institutional block-trade detection (2.5× avg volume)
22. Pullback Warning     Early exhaustion scoring (RSI div + MACD + BB re-entry + ATR)
23. Geopolitical Risk    News keyword cross-validation with technical indicators
24. News Sentiment       VADER sentiment on RSS feeds → score → signal modifier
```

---

5. API Layer

Stack: FastAPI 0.109 + Mangum (ASGI → Lambda adapter)

Endpoint	Method	Description
`/api/analyze`	GET	Analyse all configured instruments
`/api/analyze/{symbol}`	GET	Single instrument analysis
`/api/instruments`	GET	List watchlist instruments
`/api/instruments`	POST/DELETE	Add/remove instrument from watchlist
`/api/health`	GET	Lambda health check
`/api/auth/login`	GET	Initiate Google OAuth flow
`/api/auth/callback`	GET	OAuth callback — issues JWT session cookie
`/api/auth/logout`	POST	Invalidate session
`/api/auth/me`	GET	Current user profile

Lambda entry point: `app.main:handler` (Mangum wraps FastAPI app)

---

6. Frontend

Stack: Angular 17 (standalone components) · TypeScript · SCSS

Key Components

Component	Role
`AppComponent`	Root shell, market heatmap grid, polling loop
`InstrumentCardComponent`	Full analysis card for one instrument
`UserManualComponent`	In-app help + trading terminology guide
`ThemeToggleComponent`	Dark / light theme switcher (CSS variable system)

Analysis Tabs (per instrument card)

- **Signal** — Composite score gauge, multi-timeframe alignment, Expert Battle Plan, AI Executive Summary with colour-coded impact tags
- **Risk** — Validation checklist, ATR volatility regime, liquidity map, block flow, economic calendar pre-event alerts, position sizing
- **Performance** — Backtest metrics grid (Sharpe, win rate, max drawdown, MAE, expectancy)

Theme System

- CSS variable based (50+ custom properties)
- Auto-detects OS preference (`prefers-color-scheme`)
- Persists choice in `localStorage`
- Smooth 0.3s transitions across all components

---

7. Data Sources

Source	Usage	Notes
TwelveData	OHLCV historical + real-time prices	Primary market data; 4 timeframes per instrument
FMP (Financial Modelling Prep)	Economic calendar events	High-impact event pre-trade filter
Yahoo RSS	News headlines per symbol	VADER sentiment analysis
DXY / TNX symbols	Intermarket context	Fetched alongside instrument data

---

8. Testing Strategy

Test Suite (513 tests, 0 failures)

Suite	File(s)	Count	Coverage
Domain unit tests	`tests/domain/test_indicators.py`	~90	All indicator math
	`tests/domain/test_levels.py`	~80	All level calculations
	`tests/domain/test_signals.py`	~50	Scoring, filters, conflicts
	`tests/domain/test_trading.py`	~29	RVOL, ORB, position sizing
Analyzer tests	`test_analyzers.py`	4	JSON serialisation compliance
Calculation tests	`test_calculations.py`	39	Expert Battle Plan, ATR anchors, volatility
Signal generator tests	`test_signal_generator.py`	~60	Hard filters, composite score, conflict detection
Pipeline integration	`test_pipeline_integration.py`	19	Full end-to-end `analyze_instrument_lazy()`
Expert plan	`test_expert_plan.py`	~18	Battle plan section validation
Liquidity map	`test_liquidity_map.py`	~14	Cluster detection, distance calculations

Domain Test Design Principles

1. **No mocking of domain functions** — tests exercise real math
2. **Known-value fixtures** — `uptrend_closes`, `downtrend_closes`, `xau_ohlcv` defined in `conftest.py`
3. **Boundary testing** — explicit threshold boundary conditions for classifiers
4. **Floating-point tolerance** — `pytest.approx(..., rel=0.01)` for relative comparisons
5. **Identity assertions** — all boolean returns explicitly cast to `bool` (prevents `numpy.bool_` surprises)

Test Gate in CI/CD

The deploy pipeline has a **hard test gate** — no build or deploy proceeds unless `test_calculations.py` and `test_analyzers.py` pass. This protects production from calculation regressions.

```yaml
# deploy.yml
test-gate:
  steps:
    - name: Run calculation tests
      run: python -m pytest tests/test_calculations.py tests/test_analyzers.py -v
```

---

9. CI/CD Pipeline

Two Workflows

`ci.yml` — Pull Request Validation (runs on PRs to `main`)

```
lint-python          flake8 on app/ (max-line 120)
test-calculations    pytest + coverage report (≥60% threshold)
validate-terraform   terraform fmt + terraform validate
build-check          Docker build smoke test (backend + frontend)
```

`deploy.yml` — Automated Deployment (runs on push to `develop` or `main`)

```
test-gate ──────────────────────────────────────────────────────┐
    │                                                            │
setup (env=dev|production, image_tag, tf_workspace)              │
    │                                                            │
    ├── build-push-backend   Docker build → ECR push             │
    ├── build-frontend        Angular build → artifact           │
    │                                                            │
terraform-apply ◄── needs both builds                            │
    │   - S3 bucket, CloudFront, API Gateway, Lambda, DynamoDB   │
    │   - Terraform workspaces: dev / prod                       │
    │   - State in S3 + DynamoDB lock table                      │
    │                                                            │
    ├── deploy-backend    aws lambda update-function-code        │
    ├── deploy-frontend   aws s3 sync + CloudFront invalidation  │
    │                                                            │
e2e-tests    Playwright against deployed CloudFront URL ─────────┘
```

Environment Routing

Git Branch	Environment	Approval Required
`develop`	DEV	No
`main`	PRODUCTION	Yes (GitHub environment protection)

Keyless AWS Authentication (OIDC)

GitHub Actions uses **OpenID Connect** to assume an IAM role — no long-lived AWS credentials stored as secrets. The OIDC trust policy is managed in `github_oidc.tf`.

---

10. AWS Infrastructure

All infrastructure is managed as code in `infrastructure/terraform/`.

```
AWS Account
│
├── ECR (Elastic Container Registry)
│   └── market-analyser-backend   Docker images (dev:dev, prod:latest)
│
├── Lambda
│   └── market-analyser-{env}     FastAPI app via Mangum
│       ├── Runtime: Container image (Python 3.11)
│       ├── Memory: 512 MB
│       └── Timeout: 60s
│
├── API Gateway (HTTP API)
│   └── market-analyser-api-{env}
│       └── ANY /{proxy+}  →  Lambda
│
├── S3
│   ├── market-analyser-frontend-{env}   Angular SPA static files
│   └── market-analyser-tf-state         Terraform remote state
│
├── CloudFront
│   └── Distribution
│       ├── /api/*              →  API Gateway origin (no cache)
│       ├── /api/auth/callback  →  API Gateway (dedicated, zero cache)
│       └── /*                  →  S3 origin (SPA with SPA routing)
│
├── DynamoDB
│   ├── market-analyser-users-{env}    User accounts
│   └── market-analyser-tf-locks       Terraform state locking
│
└── IAM
    ├── github-actions-market-analyser  OIDC federated role (CI/CD)
    └── lambda-execution-role-{env}     Lambda + DynamoDB + ECR permissions
```

Terraform Workspaces

Workspace	Maps to	State key
`dev`	develop branch	`market-analyser/dev/terraform.tfstate`
`prod`	main branch	`market-analyser/prod/terraform.tfstate`

---

11. Security & Auth

Google OAuth 2.0 Flow

```
User → CloudFront /api/auth/login
  → Lambda → redirect to Google OAuth consent
  → Google → CloudFront /api/auth/callback
  → Lambda processes code → issues JWT cookie
  → Angular SPA reads session
```

- **JWT** signed with `JWT_SECRET_KEY` (stored in Lambda env via Terraform secret variable)
- **Session cookies** — `HttpOnly`, `Secure`, `SameSite=Lax`
- **DynamoDB** stores user profile (email, name, picture)

Secrets Management

All secrets injected at deploy time via Terraform variables sourced from GitHub Actions secrets:

```
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET
JWT_SECRET_KEY / SESSION_SECRET
TWELVEDATA_API_KEY / FMP_API_KEY / NEWS_API_KEY
```

No secrets are committed to source control or Docker images.

---

12. Branching Strategy

```
main ──────────────────────────────────────── PRODUCTION
  │
  └── develop ──────────────────────────────── DEV
        │
        ├── feature/* ─── feature branches
        ├── fix/*      ─── bug fix branches
        └── domainmodeling ── (merged ✅ Mar 2026)
```

Branch	Purpose
`main`	Production baseline; protected; requires PR + approval
`develop`	Active development; auto-deploys to DEV on push
`feature/*`	New features; PR → develop
`fix/*`	Hot fixes; PR → develop (or main for critical)

---

13. Key Design Decisions

Domain Layer as Single Source of Truth

Decision: All calculation logic lives in `backend/domain/` as pure functions. Analyzer modules are thin adapters.

Why: Before this refactor, the same ATR/RSI/Bollinger logic was duplicated across 5+ analyzer files with slight variations. A bug in one copy was never caught in others. The domain layer eliminated this, gave us 249 independently-testable units, and made the scoring engine, hard filters, and conflict detection reviewable in isolation.

Pure Functions with Primitive Types

Decision: Domain functions accept `list[float]` and return `float | dataclass`, never `pd.Series` or Pydantic models.

Why: DataFrames are a testing liability — they require mock data setup, have implicit index behaviour, and couple logic to pandas version. Primitive lists are universal, fast to instantiate in tests, and trivially serialisable.

Mangum for Lambda Compatibility

Decision: FastAPI served via Mangum rather than a custom Lambda handler.

Why: Mangum transparently adapts ASGI ↔ Lambda event format. The same FastAPI application runs locally with `uvicorn` and in production as a Lambda container — no code changes between environments.

OIDC Over IAM Access Keys

Decision: GitHub Actions authenticates to AWS via OIDC federation, not stored access keys.

Why: Eliminates the risk of long-lived credential compromise. The IAM role is scoped to only the permissions required (ECR push, Lambda update, S3 sync, CloudFront invalidation).

Terraform Workspaces for Environment Isolation

Decision: Single Terraform codebase with `dev` / `prod` workspaces.

Why: Avoids duplicating `.tf` files per environment. The workspace name is passed as a variable, allowing resource names like `market-analyser-{env}` to automatically diverge between DEV and PROD.

Composite Score Architecture

Decision: Score = `trend_score (40%) + pullback_score (30%) + strength_score (30%)`, clamped to -100..+100, with hard filters applied as a post-score gate.

Why: Keeps scoring deterministic and auditable. Hard filters (ADX, benchmark, macro event, candle confirmation) are binary gates — they don't dilute the score; they block the trade entirely. This prevents a borderline signal from being "rescued" by score arithmetic when a fundamental condition makes it untradeable.

Quick Reference

```
# Run all tests locally
cd backend
python -m pytest tests/ -q

# Run domain tests only
python -m pytest tests/domain/ -v

# Run specific suite
python -m pytest tests/test_calculations.py -v

# Local development
uvicorn app.main:app --reload --port 8000

# Docker build
```

```
docker build -t market-analyser-backend ./backend
docker run -p 8000:8000 --env-file .env market-analyser-backend
```

This document reflects the architecture as of the domain modeling merge (March 2026). The codebase has 513 passing tests, zero known regressions, and full automated deployment to AWS via GitHub Actions on every push to `develop` or `main`.